VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



NGUYEN ANH QUAN -524H0122
DANG TRI KHUONG -42300172

MID-TERM ESSAY

TOPIC 02: OPTIMIZING IMAGE LOADING, CACHING, AND DISPLAYING IN ANDROID APPLICATIONS

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



NGUYEN ANH QUAN -524H0122
DANG TRI KHUONG -42300172

MID-TERM ESSAY

TOPIC 02: OPTIMIZING IMAGE LOADING, CACHING, AND DISPLAYING IN ANDROID APPLICATIONS

Advised by
MSc. Le Van Vang

HO CHI MINH CITY, 2026

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn đến ThS. Lê Văn Vàng, giảng viên học phần Phát triển ứng dụng di động, đã hướng dẫn và cung cấp những kiến thức nền tảng để chúng em thực hiện đề tài Tối ưu tải, lưu trữ đệm và hiển thị hình ảnh trong ứng dụng Android. Những nội dung về Kotlin, bố cục XML, RecyclerView, điều hướng và kiến trúc ứng dụng là cơ sở để chúng em xây dựng ứng dụng thực nghiệm Cyclone và phân tích các kết quả đo hiệu năng.

Thông qua đề tài, chúng em có cơ hội vận dụng kiến thức đã học vào việc tải ảnh bất đồng bộ, quản lý bộ nhớ đệm, xử lý ảnh lớn và so sánh hiệu năng trước, sau tối ưu. Chúng em trân trọng sự hướng dẫn của thầy và mong nhận được góp ý để tiếp tục hoàn thiện kiến thức, kỹ năng lập trình Android.

Thành phố Hồ Chí Minh, năm 2026

Nhóm thực hiện

Ký và ghi rõ họ tên

Quan

Nguyễn Anh Quân

Khuong

Đặng Trí Khương

LỜI CAM ĐOAN

Chúng em cam đoan báo cáo Tối ưu tải, lưu trữ đệm và hiển thị hình ảnh trong ứng dụng Android được thực hiện trong khuôn khổ học phần Phát triển ứng dụng di động dưới sự hướng dẫn của ThS. Lê Văn Vàng. Báo cáo trình bày quá trình tìm hiểu, xây dựng ứng dụng Cyclone 3.2 và phân tích các phép thử trên máy ảo Pixel 5 chạy Android 12, API 31.

Các bảng số liệu thực nghiệm được tổng hợp từ dữ liệu đo của ứng dụng. Chúng em giữ nguyên các kết quả bất lợi, các trường không quan sát được và giới hạn của phép đo; không sử dụng số liệu của thiết bị khác để thay thế kết quả máy ảo. Những kiến thức và thư viện được sử dụng trong quá trình thực hiện được ghi nhận tại phần tài liệu tham khảo.

Chúng em chịu trách nhiệm về nội dung báo cáo, việc sử dụng mã nguồn và tính trung thực của các kết quả thực nghiệm được trình bày.

Thành phố Hồ Chí Minh, năm 2026

Nhóm thực hiện

Ký và ghi rõ họ tên

Quan

Nguyễn Anh Quân

Khuong

Đặng Trí Khương

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION 1

1.1 Bối cảnh thực tiễn & Sự bùng nổ nội dung đa phương tiện 1

1.2 Đặt vấn đề và Thách thức cốt lõi trong xử lý hình ảnh trên Android 1

1.3 Mục tiêu nghiên cứu và Phạm vi đề tài 2

CHAPTER 2. THEORETICAL BASIS 4

2.1 Cơ chế tải và hiển thị ảnh trên Android 4

2.2 Lập trình bất đồng bộ với Kotlin Coroutines & Quản lý vòng đời 5

2.3 Cơ chế bộ nhớ đệm đa tầng 6

2.4 Định dạng ảnh, độ phân giải và chi phí giải mã 8

2.5 Quản lý bộ nhớ đồ họa Android, ART Heap và phòng ngừa OutOfMemoryError 8

2.6 Tải ảnh theo nhu cầu và hủy tác vụ 9

2.7 Tối ưu hóa giao diện hiển thị 10

2.8 Các chiến lược tối ưu ảnh nâng cao 10

2.9 Công cụ phân tích và đo kiểm hiệu năng 11

2.10 Phương pháp luận thực nghiệm khoa học 12

CHAPTER 3. DESIGNING AND IMPLEMENTING 13

3.1 Kiến trúc giải pháp kỹ thuật Cyclone 3.2 và tích hợp thư viện Coil 2.7.0 13

3.2 Thiết kế 5 cấu hình thực nghiệm A đến E và hiện thực 8 kịch bản bắt buộc TC01 đến TC08 14

3.3 Giao diện và trạng thái xử lý ảnh 18

CHAPTER 4. EXPERIMENTAL RESULTS AND BENCHMARK ANALYSIS 20

4.1 Số liệu đo kiểm hiệu năng trước và sau tối ưu trên 36 lượt tải mới và 6 lượt tải lại 20

4.2 Đo tách riêng từng kỹ thuật trên 24 lượt và so sánh định dạng JPEG, PNG, WebP trên 9 lượt 24

4.3 Kết quả 14 ca kiểm thử chức năng, vòng đời 3.2, cuộn sâu và phân tích TC01 đến TC08 26

4.4 Phân tích chi phí xử lý và các đánh đổi hiệu năng 28

CHAPTER 5. CONCLUSION AND FUTURE WORK 30

5.1 Kết luận và đóng góp của đề tài 30

5.2 Phạm vi áp dụng và hướng phát triển 31

TÀI LIỆU THAM KHẢO 32

CHAPTER 1. INTRODUCTION

1.1 Bối cảnh thực tiễn & Sự bùng nổ nội dung đa phương tiện

Nhiều ứng dụng di động, từ thương mại điện tử, mạng xã hội đến tin tức trực tuyến, sử dụng hình ảnh làm nội dung chính. Người dùng kỳ vọng hình ảnh rõ nét, thời gian chờ ngắn và thao tác cuộn ổn định. Trên Android, sự khác nhau về phần cứng và điều kiện mạng tạo ra những thách thức khi tải, lưu trữ đệm và hiển thị hình ảnh:

- Thiết bị cấu hình thấp: Dung lượng bộ nhớ hạn hẹp và giới hạn bộ nhớ tiến trình nghiêm ngặt dễ dẫn đến tràn bộ nhớ khi giải mã ảnh có kích thước lớn.
- Thiết bị cao cấp: Màn hình tần số quét cao đòi hỏi thời gian kết xuất mỗi khung hình chỉ từ 8.33 ms đến 16.67 ms, yêu cầu quá trình tải và giải mã ảnh không được gây tắc nghẽn luồng giao diện chính.
- Môi trường mạng không ổn định: Tốc độ kết nối biến thiên từ Wi-Fi tốc độ cao đến mạng di động chập chờn kèm theo độ trễ, mất gói tin và lỗi máy chủ, đòi hỏi ứng dụng phải có cơ chế tải bất đồng bộ và lưu trữ đệm hiệu quả.

1.2 Đặt vấn đề và Thách thức cốt lõi trong xử lý hình ảnh trên Android

Tải và hiển thị hình ảnh cần phối hợp truy cập mạng, giải mã và cập nhật giao diện. Dung lượng tệp nén trên đĩa không phản ánh trực tiếp bộ nhớ của Bitmap sau giải mã. Nếu giải mã theo kích thước gốc, ảnh lớn có thể chiếm nhiều RAM; nếu hạ mẫu ngay trong quá trình giải mã theo kích thước hiển thị, bộ giải mã có thể tạo Bitmap nhỏ hơn thay vì luôn phải tạo Bitmap đầy đủ rồi mới thu nhỏ.

Nếu không được thiết kế và kiểm soát tối ưu, quá trình tải và hiển thị hình ảnh sẽ đối mặt với 4 thách thức kỹ thuật cốt lõi:

1. Nghẽn luồng giao diện chính và rớt khung hình: Việc tải dữ liệu qua mạng hoặc giải mã tệp ảnh có độ phân giải lớn tiêu tốn hàng trăm mili-giây trên CPU. Nếu thực hiện trực tiếp hoặc cập nhật dồn dập trên luồng giao diện chính, ứng dụng

sẽ vượt quá giới hạn 16.67 ms của mỗi khung hình, gây ra hiện tượng giật lag khi cuộn và dẫn đến lỗi ứng dụng không phản hồi.

2. Áp lực bộ nhớ và nguy cơ tràn bộ nhớ: Dung lượng RAM mà một bức ảnh chiếm dụng phụ thuộc hoàn toàn vào độ phân giải điểm ảnh chứ không phụ thuộc vào dung lượng tệp nén. Chẳng hạn, một bức ảnh kích thước 3000 × 3000 pixel với cấu hình màu tiêu chuẩn chiếm tới 4 byte cho mỗi điểm ảnh, tiêu tốn khoảng 34.33 MiB RAM. Việc nạp đồng thời nhiều ảnh gốc vào danh sách mà không thu nhỏ kích thước theo vùng hiển thị sẽ làm bộ nhớ tiến trình tăng vọt, kích hoạt cơ chế thu gom rác liên tục và dễ gây lỗi tràn bộ nhớ.
3. Lãng phí băng thông mạng và thời gian tải lại: Khi thiếu cơ chế bộ nhớ đệm trên RAM và bộ nhớ lưu trữ cùng chiến lược sử dụng ảnh thu nhỏ cho danh sách, ứng dụng buộc phải tải lại toàn bộ tệp ảnh gốc có dung lượng lớn mỗi khi người dùng cuộn qua lại hoặc mở lại màn hình.
4. Xung đột vòng đời và hiển thị sai lệch dữ liệu: Trong quá trình cuộn nhanh danh sách có cơ chế tái sử dụng thành phần giao diện hoặc khi xoay màn hình, các thành phần hiển thị liên tục được gắn vào dữ liệu mới. Nếu ứng dụng không kịp thời hủy các tác vụ tải ảnh không còn cần thiết, hình ảnh của mục cũ khi tải xong muộn sẽ ghi đè và hiển thị sai vào vị trí của mục sản phẩm mới.

1.3 Mục tiêu nghiên cứu và Phạm vi đề tài

Nhằm giải quyết các bài toán kỹ thuật đặt ra và đáp ứng đầy đủ yêu cầu của đề tài, nghiên cứu tập trung phát triển ứng dụng thực nghiệm Cyclone phiên bản 3.2 với các mục tiêu cụ thể:

1. Nghiên cứu cơ sở lý thuyết: Hệ thống hóa các nội dung trọng tâm về chu trình tải và hiển thị ảnh, lập trình bất đồng bộ với Kotlin Coroutines, cơ chế lưu trữ đệm đa tầng từ RAM đến bộ nhớ lưu trữ, các định dạng ảnh, kỹ thuật quản lý bộ nhớ Bitmap, tải ảnh theo nhu cầu cùng các công cụ đo lường hiệu năng giao diện trên Android.
2. Thiết kế và phát triển ứng dụng thực nghiệm: Xây dựng danh sách sản phẩm dạng lưới so le hai cột bằng Kotlin, XML và RecyclerView, kết hợp ViewModel, LiveData và Coil. Các cấu hình A đến E dùng chung giao diện

và dữ liệu để đối chiếu tác động của bộ nhớ đệm, giải mã theo khung nhìn và ảnh thu nhỏ.

3. Thực nghiệm và kiểm thử: Đo tải lần đầu, tải lại khi có bộ nhớ đệm, CPU, PSS và độ trễ khung hình ở quy mô 100, 500 và 1.000 mục trên máy ảo; kiểm tra cuộn nhanh, ảnh lỗi, mất kết nối tới nguồn thử, xoay màn hình và khôi phục trạng thái.

Phạm vi đề tài: Đề tài tập trung chuyên sâu vào bài toán kỹ thuật tải ảnh bất đồng bộ, cơ chế bộ nhớ đệm, tối ưu giải mã ảnh và đo kiểm hiệu năng hiển thị trên nền tảng Android. Hệ thống được xây dựng bám sát kiến trúc Kotlin, XML và RecyclerView, không mở rộng sang các chức năng nghiệp vụ thương mại điện tử như giỏ hàng hay thanh toán để đảm bảo tính tập trung và độ tin cậy của các phép đo thực nghiệm.

CHAPTER 2. THEORETICAL BASIS

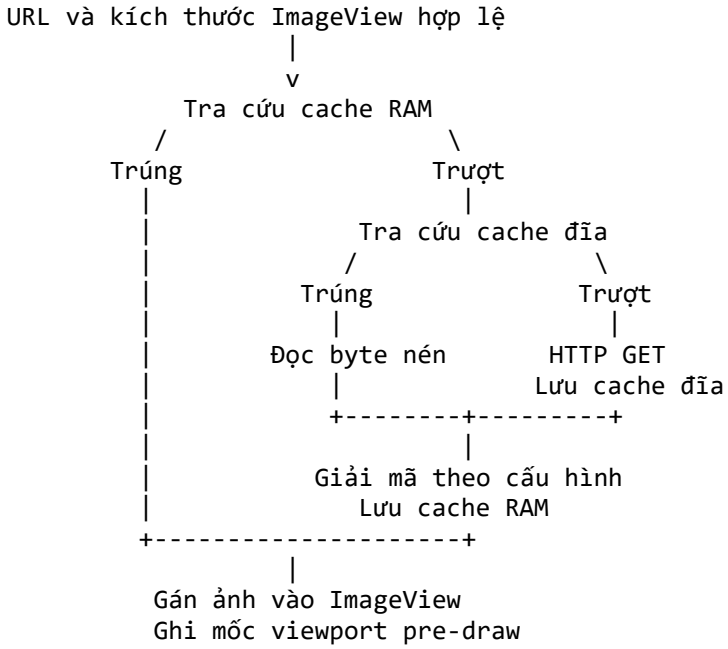
2.1 Cơ chế tải và hiển thị ảnh trên Android

Quá trình lấy ảnh từ địa chỉ mạng và hiển thị lên giao diện Android là chuỗi tương tác qua nhiều tầng hệ thống gồm mạng, tệp tin, thư viện đồ họa C++ Skia, máy ảo ART và bộ xử lý đồ họa GPU. Trong kiến trúc Cyclone 3.2 sử dụng Coil 2.7.0, chu trình này gồm 5 giai đoạn:

Quy trình 5 giai đoạn xử lý ảnh trên Android và áp dụng trong Cyclone 3.2:

1. Giai đoạn 1 - Tiếp nhận URL, kiểm tra kích thước khung nhìn và tra cứu bộ nhớ đệm RAM: Lớp ProductRepository sinh địa chỉ URL theo cấu hình, ví dụ http://127.0.0.1:8000/seed/item1/480/720?format=JPEG&v=1. Trước khi nạp, ProductAdapter chờ ImageView hoàn tất đo đạc với chiều rộng và chiều cao lớn hơn 0. Thư viện Coil kiểm tra khóa trong bộ nhớ đệm RAM; nếu trúng bộ nhớ đệm, đối tượng Bitmap đã giải mã được trả về tức thì mà không cần đọc đĩa, tải mạng hay giải mã lại.
2. Giai đoạn 2 - Tra cứu bộ nhớ đệm đĩa và nhận dữ liệu mạng: Nếu trượt RAM, Coil tìm tệp ảnh nén trong thư mục cacheDir/image_cache_*. Nếu tiếp tục trượt hoặc cấu hình A tắt bộ nhớ đệm, OkHttp thực hiện yêu cầu HTTP GET với connectTimeout 10 giây, readTimeout 15 giây và callTimeout 30 giây. Các thao tác vào ra được thư viện thực hiện bất đồng bộ, tránh chặn luồng giao diện.
3. Giai đoạn 3 - Xác định kích thước giải mã: A và B yêu cầu Size.ORIGINAL. C, D và E dùng kích thước hiển thị để hạ mẫu trong quá trình giải mã. Việc đổi kích thước giải mã và việc đổi URL sang ảnh thu nhỏ là hai kỹ thuật khác nhau: C vẫn nhận tệp nguồn lớn nhưng tạo Bitmap nhỏ hơn.
4. Giai đoạn 4 - Giải mã dữ liệu nén và lưu kết quả: Coil sử dụng bộ giải mã phù hợp để chuyển dữ liệu JPEG, PNG hoặc WebP thành Bitmap theo kích thước đã xác định. Khi bật cache, tệp nén có thể được lưu trên đĩa và kết quả đã giải mã được lưu trong RAM để phục vụ yêu cầu sau.
5. Giai đoạn 5 - Cập nhật ImageView, ghi mốc chuẩn bị vẽ và kết xuất khung hình: Đối tượng Drawable chứa Bitmap được chuyển về luồng chính Main Thread để

gán vào ImageView, thay thế ảnh chờ bg_placeholder hoặc hiển thị ảnh lỗi ic_error nếu yêu cầu thất bại. Ứng dụng ghi nhận mốc chuẩn bị vẽ qua ViewTreeObserver.OnPreDrawListener trước khi RenderThread và SurfaceFlinger hợp thành khung hình xuất ra màn hình theo chu kỳ VSYNC.



Ranh giới các mốc thời gian: fetchMs phản ánh khoảng tải dữ liệu của yêu cầu; decodeMs phản ánh giải mã; totalMs là thời gian yêu cầu từ lúc bắt đầu tới khi kết thúc. viewportPreDrawMs được ghi trước lượt vẽ của vùng nhìn. Mốc pre-draw không phải thời điểm hệ thống hoàn tất hợp thành khung hình hoặc điểm ảnh thực sự xuất hiện trên màn hình.

2.2 Lập trình bất đồng bộ với Kotlin Coroutines & Quản lý vòng đời

Android vận hành giao diện theo mô hình luồng đơn, trong đó Main Thread đảm nhiệm xử lý cảm ứng, đo đạc, bố cục và vẽ khung hình. Nếu đọc ghi đĩa hoặc giải mã ảnh lớn trực tiếp trên Main Thread, thời gian xử lý sẽ vượt quá chu kỳ VSYNC 16.67 ms trên màn hình 60Hz, gây trễ hạn khung hình và dẫn đến lỗi ứng dụng không phản hồi ANR nếu tắc nghẽn quá 5 giây.

Bộ điều phối	Bản chất thực thi luồng	Cách áp dụng cụ thể trong mã nguồn Cyclone 3.2
Dispatchers.Main	Luồng giao diện chính cập nhật hiển thị	Quan sát LiveData chứa GalleryState, gắn ProductAdapter, đặt ảnh chờ hoặc ảnh lỗi và cập nhật ImageView sau khi giải mã.
Dispatchers.Default	Nhóm luồng tối ưu cho tính toán CPU	Được ProductViewModel.rebuild dùng để khởi tạo danh sách 100, 500 hoặc 1.000 đối tượng Product cùng tỷ lệ khung hình và URL mà không làm khựng giao diện.
Dispatchers.IO	Nhóm luồng co giãn cho vào ra đĩa và mạng	Xóa bộ nhớ đệm đĩa qua clearCache, ghi gói kết quả CSV, JSON, ZIP trong ExperimentFragment, lấy mẫu Debug.MemoryInfo định kỳ và chạy tải mạng cùng giải mã của Coil và OkHttp.
viewModelScope & lifecycleScope	Phạm vi quản lý đồng thời theo vòng đời	Gắn tác vụ bất đồng bộ với vòng đời của ProductViewModel và viewLifecycleOwner của Fragment; tự động hủy tác vụ khi màn hình bị hủy nhằm tránh rò rỉ bộ nhớ.

Lưu ý kỹ thuật cốt lõi: Từ khóa suspend hoặc việc khởi chạy coroutine trên Dispatchers.Main không tự biến tác vụ nặng thành nhẹ. Các tác vụ mạng, đọc ghi đĩa và giải mã bắt buộc phải chuyển sang luồng nền Dispatchers.IO và Dispatchers.Default đúng như cách Coil và ProductViewModel thực hiện trong Cyclone 3.2.

2.3 Cơ chế bộ nhớ đệm đa tầng

Bộ nhớ đệm lưu tạm kết quả đã xử lý để phục vụ các lần truy cập sau với chi phí thấp hơn. Trong kiến trúc Android và đề tài Cyclone 3.2, hệ thống phân biệt rõ 3 tầng bộ nhớ đệm:

Tiêu chí	Tầng 1: Bộ nhớ đệm RAM MemoryCache	Tầng 2: Bộ nhớ đệm đĩa DiskCache	Tầng 3: Bộ nhớ đệm HTTP, máy chủ và CDN
Dữ liệu lưu trữ	Đối tượng Bitmap đã giải mã sẵn sàng hiển thị.	Tệp ảnh nén JPEG, PNG hoặc WebP lưu trong thư mục context.cacheDir.	Bản sao phản hồi HTTP theo chuẩn RFC 9111 dựa trên Cache-Control, ETag và Last-Modified.

Tiêu chí	Tầng 1: Bộ nhớ đệm RAM MemoryCache	Tầng 2: Bộ nhớ đệm đĩa DiskCache	Tầng 3: Bộ nhớ đệm HTTP, máy chủ và CDN
Tốc độ và chi phí	Truy xuất Bitmap đã giải mã, tránh đọc mạng và giải mã lại. Giá trị 0 ms trong log là kết quả ở độ phân giải mili giây, không có nghĩa thao tác không tốn thời gian.	Tốc độ trung bình, không tốn băng thông mạng nhưng vẫn cần đọc đĩa và giải mã lại thành Bitmap.	Phụ thuộc tốc độ mạng hoặc kiểm tra độ tươi mới với mã phản hồi 304 từ máy chủ.
Cấu hình trong Cyclone 3.2	Giới hạn 25% bộ nhớ ứng dụng qua maxSizePercent bằng 0.25 cho các cấu hình B, C, D, E; cấu hình A tắt hoàn toàn.	Giới hạn 150 MiB cho mỗi cấu hình B, C, D, E tại các thư mục riêng từ image_cache_B đến image_cache_E để tránh nhiễm chéo.	Không cấu hình bộ nhớ đệm HTTP riêng trên OkHttp; máy chủ cục bộ chạy tại 127.0.0.1:8000 qua cầu nối adb reverse và không dùng CDN.
Vòng đời tồn tại	Mất toàn bộ khi tiến trình dừng hoặc khi phần tử cũ bị loại bỏ theo thuật toán LRU lúc bộ nhớ đầy.	Tồn tại bền vững qua các lần khởi động lại ứng dụng, có thể bị hệ điều hành thu hồi khi thiếu bộ nhớ lưu trữ.	Tuân theo chính sách tiêu đề phản hồi của máy chủ hoặc cấu hình kiểm soát của trình khách.

Chính sách kiểm soát ảnh thay đổi trên máy chủ trong Cyclone 3.2: Máy chủ cục bộ gửi tiêu đề Cache-Control: no-store, nhưng các bộ nạp ImageLoader có bật bộ nhớ đệm của Cyclone 3.2 được đặt thuộc tính respectCacheHeaders bằng false nhằm cô lập và kiểm chứng chính xác hoạt động của bộ nhớ đệm RAM và đĩa mà không bị nhiễu bởi tầng HTTP. Do đó, nếu máy chủ đổi nội dung tệp ảnh nhưng ứng dụng vẫn gọi địa chỉ cũ có hậu tố ?v=1, ứng dụng sẽ trả về ảnh cũ từ bộ nhớ đệm. Để nhận ảnh mới, ứng dụng sử dụng cơ chế đổi khóa phiên bản trên địa chỉ URL từ ?v=1 sang ?v=2 hoặc xóa bộ nhớ đệm trước khi tải lại như đã kiểm chứng ở kịch bản TC11. Thao tác đổi phiên bản URL trong ứng dụng chỉ đổi khóa truy vấn

chứ không tự sửa tệp trên máy chủ, đồng thời ứng dụng hiện chưa triển khai xác thực lại tự động bằng ETag hay Last-Modified.

2.4 Định dạng ảnh, độ phân giải và chi phí giải mã

Nguyên lý nền tảng trong xử lý đồ họa Android là phân biệt rõ dung lượng tệp nén trên đĩa hoặc mạng, độ phân giải điểm ảnh và dung lượng Bitmap chiếm dụng trong RAM:

- JPEG: Nén có suy hao dựa trên biến đổi cosin rời rạc DCT; phù hợp với ảnh chụp phức tạp, dung lượng tệp vừa phải, tốc độ giải mã trên CPU rất nhanh nhưng không hỗ trợ kênh trong suốt Alpha.
- PNG: Nén bảo toàn dữ liệu dựa trên thuật toán DEFLATE và hỗ trợ kênh Alpha; dung lượng tệp thường lớn hơn đáng kể so với JPEG trên cùng nội dung ảnh phức tạp.
- WebP: Định dạng hiện đại của Google hỗ trợ cả nén suy hao, nén bảo toàn và kênh Alpha. Trên bộ dữ liệu thử nghiệm Cyclone 3.2 cùng tham số quality bằng 88 khi sinh ảnh, WebP cho dung lượng tệp nhỏ nhất, giảm khoảng 45% so với JPEG và khoảng 75% so với PNG, nhưng thời gian giải mã decodeMs và thời gian CPU tiến trình có thể cao hơn JPEG đôi chút. Do đó, tệp nén nhỏ hơn không đồng nghĩa với giải mã nhanh hơn trên CPU, và cùng chỉ số quality bằng 88 giữa hai bộ mã hóa không đảm bảo chất lượng thị giác ngang nhau tuyệt đối.

2.5 Quản lý bộ nhớ đồ họa Android, ART Heap và phòng ngừa OutOfMemoryError

Khi giải mã tệp ảnh nén sang đối tượng Bitmap với cấu hình mặc định ARGB_8888 dùng 4 byte cho mỗi điểm ảnh gồm 4 kênh Alpha, Red, Green và Blue, dung lượng bộ nhớ lý thuyết trước khi tính chi phí căn chỉnh hàng và tiêu đề đối tượng được ước tính theo công thức:

Bộ nhớ Bitmap tính bằng byte ≈ Chiều rộng điểm ảnh × Chiều cao điểm ảnh × 4 byte

Ví dụ lý thuyết: Một ảnh vuông kích thước gốc 3000 × 3000 pixel giải mã nguyên bản chiếm khoảng 3000 × 3000 × 4 = 36.000.000 byte, tương đương 34.33

MiB RAM; nếu thu nhỏ về 720 × 720 pixel chỉ chiếm khoảng 1.98 MiB RAM, và giải mã theo kích thước thực của ô hiển thị còn nhỏ hơn nữa. Giải mã hàng loạt ảnh gốc không chỉ làm tăng vọt bộ nhớ tiến trình, gây nguy cơ tràn bộ nhớ OutOfMemoryError, mà còn buộc bộ thu gom rác ART GC phải hoạt động liên tục.

Lưu ý đọc chỉ số bộ nhớ trong Cyclone 3.2: Chỉ số sampledPssPeakKb quy đổi ra MiB bằng cách chia cho 1024 là giá trị đỉnh của bộ nhớ PSS toàn tiến trình lấy mẫu định kỳ từ 500 ms trở lên, bao gồm Java Heap, Native Heap, bộ nhớ đồ họa và phần bộ nhớ dùng chung chia theo tỷ lệ. Vì vậy, không được cộng tổng số byte của mọi Bitmap trong lịch sử yêu cầu để suy ra RAM đỉnh. Tương tự, tổng thời gian thu gom rác gc_ms từ Debug.getRuntimeStats bao gồm cả các pha thu gom chạy đồng thời ở nền, không phải thời gian dừng cứng Main Thread.

2.6 Tải ảnh theo nhu cầu và hủy tác vụ

Kỹ thuật tải ảnh theo nhu cầu đảm bảo ứng dụng chỉ khởi tạo yêu cầu tải và giải mã cho các phần tử nằm trong vùng nhìn thấy cùng vùng đệm nhỏ lân cận. Khi danh sách có 1.000 mục, ứng dụng chỉ giữ 1.000 đối tượng dữ liệu Product nhẹ chứa mã định danh, tên, giá và chuỗi URL, tuyệt đối không nạp sẵn 1.000 Bitmap vào RAM cùng lúc.

Khi cuộn danh sách, các ô rời khỏi màn hình được RecyclerView đưa vào vùng tái sử dụng để gắn cho mục mới. Trong onBindViewHolder và onViewRecycled, lớp ProductAdapter của Cyclone 3.2 thực thi đồng thời 2 cơ chế bảo vệ:

1. Gọi binding.thumb.dispose để hủy ngay yêu cầu tải và giải mã ảnh cũ không còn cần thiết.
2. Tăng biến đếm thế hệ token và gỡ bộ lắng nghe bố cục cũ. Các callback chỉ được ghi nhận khi token còn khớp với mục đang được gắn. Kết hợp với cơ chế hủy yêu cầu của Coil, cách này tránh nhận kết quả của lượt gắn dữ liệu cũ khi ViewHolder đã được tái sử dụng.

2.7 Tối ưu hóa giao diện hiển thị

Bám sát học phần phát triển ứng dụng di động gồm bài giảng *Android RecyclerView*, *Lesson 5 Layouts*, *Lesson 7 Navigation* và *Lesson 8 UI Layer*, ứng dụng Cyclone 3.2 áp dụng các kỹ thuật tối ưu giao diện sau:

- Tái sử dụng ViewHolder và lưới so le hai cột: Dùng RecyclerView với StaggeredGridLayoutManager hai cột dọc, bật setHasStableIds và setHasFixedSize. Bố cục thẻ sản phẩm trong item_product.xml dùng ConstraintLayout phẳng một cấp, cố định trước tỷ lệ khung hình dimensionRatio theo 5 tỷ lệ 2:3, 1:1, 3:2, 3:4 và 4:3 để giao diện không bị nhảy bố cục khi ảnh tải xong.
- Tránh yêu cầu đo đạc bố cục dư thừa ở bản 3.2: Khi một ViewHolder được tái sử dụng cho sản phẩm mới có cùng tỷ lệ khung hình với sản phẩm trước đó, ProductAdapter không gọi lại requestLayout dư thừa nhưng vẫn chờ kích thước ImageView hợp lệ trước khi gửi yêu cầu tải ảnh.
- Quản lý trạng thái và điều hướng an toàn: Dùng ProductViewModel và LiveData chứa GalleryState để đồng bộ cấu hình và dữ liệu; dùng Navigation Component với Safe Args chỉ truyền số nguyên productId sang màn hình chi tiết mà không truyền Bitmap lớn qua Bundle; đồng thời lưu vị trí cuộn qua onSaveInstanceState kèm khóa nội dung nhằm khôi phục đúng vùng đang xem sau khi quay lại, xoay màn hình hoặc khi tiến trình bị hủy ở nền.

2.8 Các chiến lược tối ưu ảnh nâng cao

Để phân tích rõ đóng góp của từng chiến lược tối ưu thay vì gộp chung, cần phân biệt cơ chế tác động của từng kỹ thuật:

Giải mã theo vùng hiển thị: C, D và E truyền kích thước của ImageView vào yêu cầu Coil. Phép đo B/C trên cùng URL ảnh gốc cho thấy kỹ thuật này giảm PSS và thời gian giải mã trong điều kiện thử nghiệm. Nếu giữ URL ảnh gốc như C, dung lượng tệp nguồn vẫn không giảm.

Ảnh thu nhỏ cho danh sách và ảnh lớn cho chi tiết: D và E dùng ảnh cạnh dài 720 px cho danh sách, ảnh cạnh dài 1.600 px cho chi tiết và giải mã theo khung nhìn.

Với 5 ảnh đầu trong bộ dữ liệu, tổng dung lượng tệp tương ứng các yêu cầu NETWORK thành công giảm 85,50% so với ảnh gốc cạnh dài 3.000 px.

- Ảnh chờ, ảnh lỗi dự phòng và biến thể trải nghiệm thị giác: Hiển thị nền chờ bg_placeholder khi đang tải và biểu tượng ic_error khi lỗi mạng, lỗi HTTP 404 hoặc quá thời gian chờ. Cấu hình E bổ sung hiệu ứng chuyển cảnh mờ dần 200 ms và dùng ảnh thu nhỏ sẵn có trong bộ nhớ đệm RAM làm ảnh xem trước cho màn hình chi tiết; đây là biến thể trải nghiệm thị giác và có thể tốn thêm chi phí vẽ khung hình so với cấu hình D.

2.9 Công cụ phân tích và đo kiểm hiệu năng

Hệ thống đo kiểm trong Cyclone 3.2 kết hợp bộ thu thập nội bộ gồm ImageLoadRecorder.kt và MeasurementSession.kt với các công cụ chuẩn của Android:

- Bộ lắng nghe Coil EventListener: Ghi nhận thời gian đơn điệu từ System.nanoTime cho từng yêu cầu gồm tổng thời gian totalMs từ lúc bắt đầu đến khi thành công, lỗi hoặc hủy, thời gian tải mạng fetchMs, thời gian giải mã decodeMs, nguồn trả về NETWORK, DISK hoặc MEMORY_CACHE và kích thước Bitmap. Các yêu cầu chưa kết thúc khi dừng phiên đo được giữ nguyên trạng thái incomplete, không gán bằng 0 ms.
- Lấy mẫu bộ nhớ định kỳ và bộ đếm ART GC: Lấy mẫu Debug.MemoryInfo định kỳ từ 500 ms trở lên trên luồng nền để tìm đỉnh bộ nhớ sampledPssPeakKb; đồng thời thu thập mức chênh lệch từ Debug.getRuntimeStats gồm số lần thu gom rác gc_count, tổng thời gian gc_ms và tổng số byte cấp phát allocated_bytes.
- Theo dõi độ trễ khung hình qua FrameMetrics từ API 31: Dùng Window.OnFrameMetricsAvailableListener đo tổng thời gian khung hình TOTAL_DURATION, tính phân vị thứ 90 frame_p90_ms, tỷ lệ khung hình trễ hạn khi TOTAL_DURATION lớn hơn hoặc bằng DEADLINE sau khi bỏ khung hình đầu tiên, cùng thời gian các pha đo đạc layout_p90_ms, vẽ draw_p90_ms và đồng bộ GPU sync_p90_ms.

• Theo dõi CPU tiến trình và dấu vết lấy mẫu hàm ART: Ghi nhận tổng thời gian CPU của các luồng trong tiến trình qua Process.getElapsedCpuTime thành processCpuMs. Do Perfetto trên máy ảo thiếu dữ liệu lập lịch hạt nhân, nghiên cứu đã thu thập 2 bản dấu vết ART CPU Method Trace phiên bản 3 theo đồng hồ kép riêng cho cấu hình A và D để kiểm chứng hoạt động giải mã trên các luồng.

2.10 Phương pháp luận thực nghiệm khoa học

Các phép đo trong hồ sơ Cyclone 3.2 tuân thủ nghiêm ngặt nguyên tắc kiểm soát biến số nhằm đảm bảo tính khách quan và khả năng tái lập:

• Cố định môi trường và bộ dữ liệu: Thực nghiệm tiến hành trên máy ảo Pixel_5 AVD chạy Android 12 mức API 31, kiến trúc x86_64, màn hình 1080 × 2340 với bộ kết xuất GLES NVIDIA GeForce GTX 1650, dùng bản dựng Cyclone-3.2-benchmark.apk không bật cờ gỡ lỗi. Máy chủ HTTP cục bộ phục vụ bộ dữ liệu cố định gồm 9.000 tệp v1 tại datasets/fixtures và 300 tệp v2 tại datasets/updated-v2 đã kiểm định mã băm SHA-256 qua cầu nối adb reverse tcp:8000 tcp:8000.

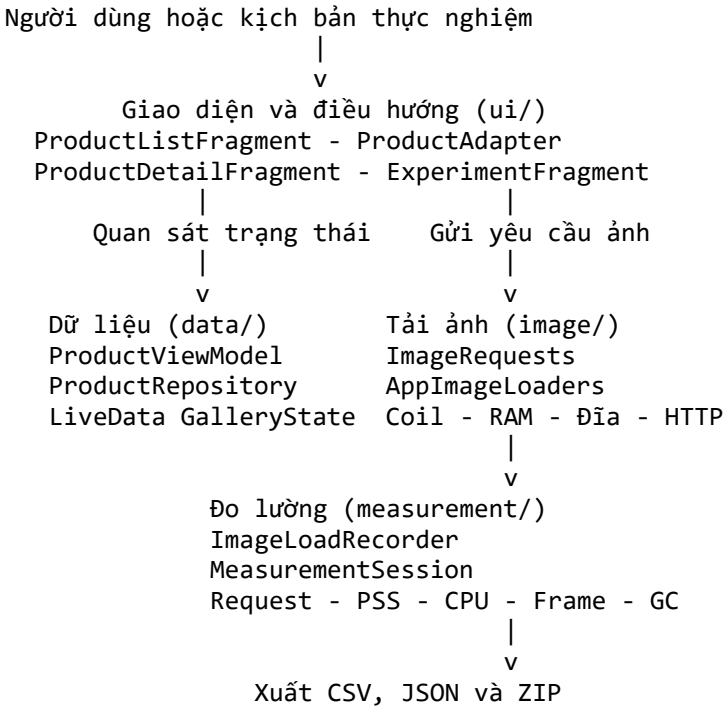
• Đo lặp và đảo thứ tự luân phiên: Mỗi cặp kịch bản trong ma trận chính được đo lặp n = 3 lượt. Trước mỗi lượt tải mới, kịch bản tự động xóa sạch bộ nhớ đệm thử nghiệm và khởi động lại tiến trình. Thứ tự chạy giữa cấu hình A và D được đảo luân phiên qua các vòng lặp nhằm giảm sai số môi trường.

Quy tắc thống kê: Ma trận chính có 36 lượt cold, tương ứng 2 cấu hình × 3 quy mô × 2 kịch bản × 3 lần lặp; nhóm tải lại có 6 lượt sau 6 lượt nạp mồi riêng. Mỗi chỉ số tổng hợp là trung vị các giá trị quan sát. Các lượt lấy mẫu CPU, kiểm thử chức năng và nạp mồi được phân nhóm riêng; 114 phiên được lập chỉ mục không được xem là 114 lượt đối chứng A/D độc lập.

CHAPTER 3. DESIGNING AND IMPLEMENTING

3.1 Kiến trúc giải pháp kỹ thuật Cyclone 3.2 và tích hợp thư viện Coil 2.7.0

Ứng dụng thực nghiệm chính thức có tên hiển thị Cyclone, tên gói com.example.imageopt, phiên bản 3.2 với versionCode 5, lưu trữ tại kho Git công khai https://github.com/Sunn-dk/Cyclone ở thẻ v3.2, mã commit f797efb kế thừa từ commit nền d9f101c. Mã nguồn được tổ chức thành 4 phân hệ dưới thư mục Cyclone/app/src/main/java/com/example/imageopt/:



Các quyết định kỹ thuật trong mô-đun tải ảnh và bộ nhớ đệm:

1. Duy trì một bộ nạp ImageLoader hoạt động trong ImageLoaderFactory.kt: Lớp AppImageLoaders chỉ giữ một ImageLoader ứng với cấu hình đang chọn; khi chuyển cấu hình, bộ nạp cũ được gọi shutdown ngay lập tức nhằm tránh 5 cấu hình chiếm giữ đồng thời 5 vùng bộ nhớ đệm RAM 25%.
2. Tách biệt nguồn ảnh và kích thước giải mã trong ImageRequests.kt ở bản 3.2: Ở màn hình danh sách với loadList, cấu hình D và E chọn địa chỉ ảnh thu nhỏ p.thumbUrl cạnh dài 720px, còn A, B và C dùng ảnh gốc p.rawUrl cạnh dài 3000px. Ở màn hình chi tiết với loadDetail, bản 3.2 chuẩn hóa để cả D và E

cùng dùng ảnh chi tiết p.detailUrl cạnh dài 1600px thay vì ảnh gốc 3000px, đồng thời giải mã theo kích thước thực của khung hiển thị.

Đoạn mã dưới đây là hàm loadList trong ImageRequests.kt của phiên bản 3.2. ExperimentConfig là kiểu cấu hình A đến E; hàm gửi yêu cầu tới Coil và thông báo kết quả qua onResult.

```
fun loadList(view: ImageView, p: Product, config: ExperimentConfig,
        loader: ImageLoader,
             onResult: (Boolean) -> Unit = {}) {
    val url = if (config.thumbInList) p.thumbUrl else p.rawUrl
    view.load(url, loader) {
        if (config.originalSize) size(Size.ORIGINAL)
        else size(view.width.coerceAtLeast(1),
        view.height.coerceAtLeast(1))
        if (!config.useCache) {
            memoryCachePolicy(CachePolicy.DISABLED)
            diskCachePolicy(CachePolicy.DISABLED)
        }
        placeholder(R.drawable.bg_placeholder)
        crossfade(if (config.polish) 200 else 0)
        if (config.polish) memoryCacheKey(url)
        error(R.drawable.ic_error)
        listener(onSuccess = { _, _ -> onResult(true) }, onError = { _, _
        -> onResult(false) })
    }
}
```

3.2 Thiết kế 5 cấu hình thực nghiệm A đến E và hiện thực 8 kịch bản bắt buộc TC01 đến TC08

Để tái lập hai cấu hình trước và sau tối ưu trên cùng giao diện và nghiệp vụ, Cyclone 3.2 tích hợp 5 cấu hình thực nghiệm từ A đến E trên cùng mã nguồn và tệp APK. Hai tệp Cyclone-3.2-debug.apk và Cyclone-3.2-benchmark.apk là hai loại bản dựng phục vụ gỡ lỗi và đo hiệu năng, không phải hai phiên bản trước và sau tối ưu:

Cấu hình	Nguồn ảnh danh sách và chi tiết	Chính sách giải mã	Bộ nhớ đệm RAM và đĩa	Mục đích trong thiết kế thực nghiệm
A - Cơ sở	Danh sách: Ảnh gốc 3000px Chi tiết: Ảnh gốc 3000px	Giải mã nguyên gốc Size.ORIGINAL	Tắt hoàn toàn cả hai tầng bộ nhớ đệm	Phiên bản cơ sở trước tối ưu để đối chứng với D trong ma trận chính và so sánh cặp với B; vẫn tải bất đồng bộ qua Coil.

Cấu hình	Nguồn ảnh danh sách và chi tiết	Chính sách giải mã	Bộ nhớ đệm RAM và đĩa	Mục đích trong thiết kế thực nghiệm
B - Bộ đệm	Danh sách: Ảnh gốc 3000px Chi tiết: Ảnh gốc 3000px	Giải mã nguyên gốc Size.ORIGINAL	Bật RAM 25% ứng dụng và đĩa 150 MiB	So sánh cặp A và B khi tải lại sau bước nạp mồi nhằm cô lập tác động riêng của bộ nhớ đệm.
C - Hạ mẫu	Danh sách: Ảnh gốc 3000px Chi tiết: Ảnh gốc 3000px	Giải mã thu nhỏ theo ô hiển thị	Bật RAM 25% ứng dụng và đĩa 150 MiB	So sánh cặp B và C khi tải mới trên cùng URL ảnh gốc 3000px nhằm cô lập tác động của giải mã theo vùng hiển thị.
D - Tối ưu	Danh sách: Ảnh nhỏ 720px Chi tiết: Ảnh chi tiết 1600px	Giải mã thu nhỏ theo ô hiển thị	Bật RAM 25% ứng dụng và đĩa 150 MiB	Phiên bản tối ưu chính thức để đối chứng với A trong ma trận chính và so sánh với C nhằm tách tác động của ảnh thu nhỏ.
E - UX	Danh sách: Ảnh nhỏ 720px Chi tiết: Ảnh chi tiết 1600px	Giải mã theo ô hiển thị kèm mờ dần 200 ms	Bật bộ đệm và dùng ảnh RAM làm ảnh xem trước	Biến thể hoàn thiện trải nghiệm thị giác đã kiểm chứng chức năng và tách khỏi phép đo tốc độ thuần của D.

Bảng ánh xạ 8 kịch bản thực nghiệm bắt buộc từ TC01 đến TC08 của đề bài với bộ kiểm thử tự động trong Cyclone 3.2:

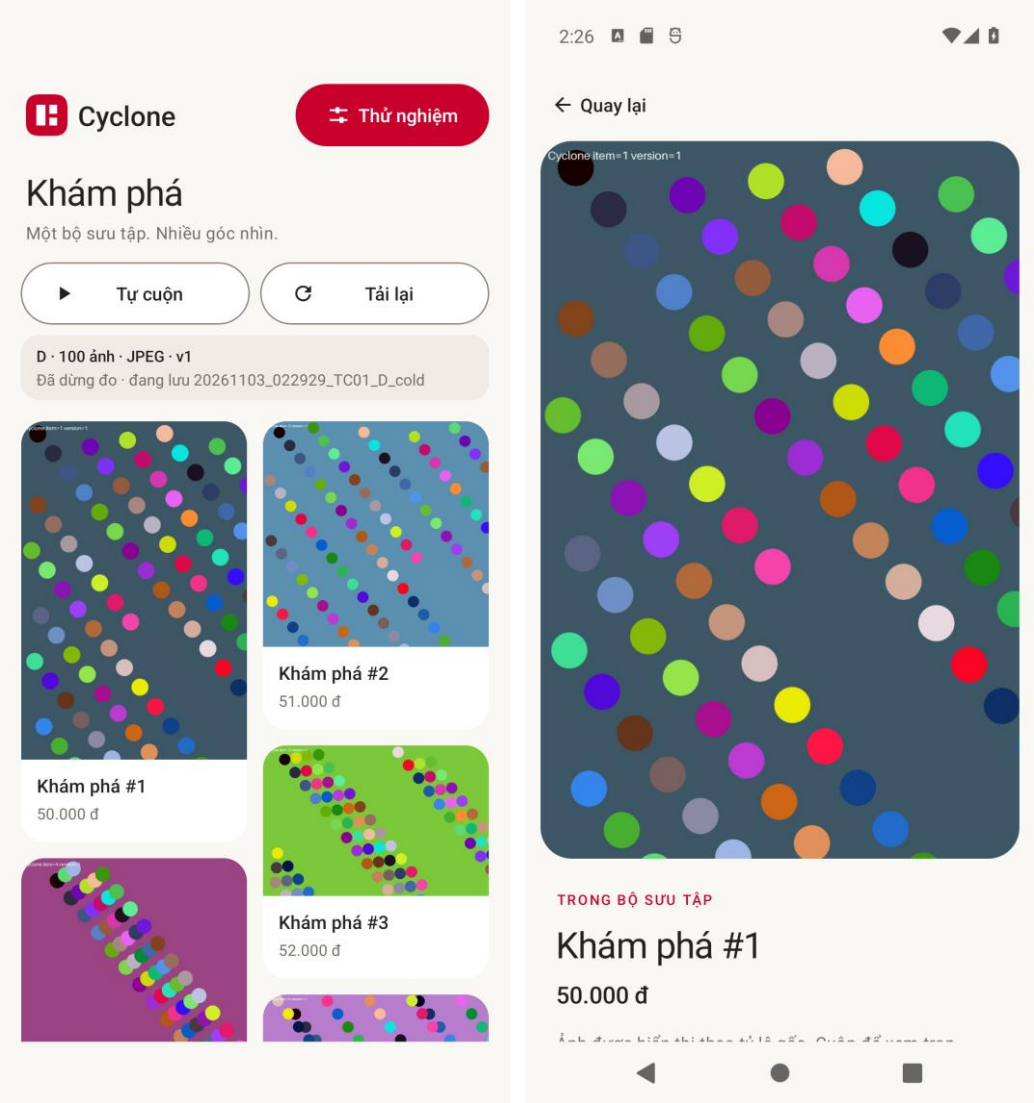
Mã TC	Tên kịch bản bắt buộc	Cách hiện thực và mã ca chạy trong hồ sơ Cyclone 3.2	Chỉ số thu thập và kỳ vọng kiểm chứng
TC01	Mở danh sách ảnh lần đầu	Chạy ca TC01_D_cold cùng các cặp TC08_100, TC08_500, TC08_1000 ở chế độ load qua 3 lượt lặp cho A và D sau khi xóa bộ nhớ đệm và khởi động lại.	Đo viewport_ms, request_ms, decode_ms và đỉnh bộ nhớ sampled_peak_pss_mib; xác nhận nguồn trả về từ NETWORK bằng 5.

Mã TC	Tên kịch bản bắt buộc	Cách hiện thực và mã ca chạy trong hồ sơ Cyclone 3.2	Chỉ số thu thập và kỳ vọng kiểm chứng
TC02	Mở lại khi bộ nhớ đệm còn hiệu lực	Chạy TC02_D_RAM trong cùng tiến trình, TC02_D_disk khi xóa RAM giữ đĩa, TC10_D_restart khi khởi động lại tiến trình, cùng 6 lượt reload/100 và cặp đo cache_warm.	Xác minh nguồn trả về từ MEMORY_CACHE đạt 0-0-5 và từ DISK đạt 0-5-0; thời gian hiển thị giảm mạnh so với lần tải đầu.
TC03	Cuộn nhanh qua nhiều ảnh	Chạy ma trận cuộn 6 vùng nhìn trong 8 giây ở quy mô 100, 500, 1.000 mục qua 3 lượt lặp cho A và D, kèm ca TC03_scroll_visual, TC03_cancel_pressure và D_traverse_1000.	Đo frame_p90_ms, tỷ lệ trễ hạn deadline_miss_pct và số lần gc_count; kiểm chứng hủy yêu cầu CANCEL và không gắn nhầm ảnh cũ.
TC04	Thử nghiệm ảnh phân giải lớn và định dạng	Chạy 24 lượt đo tách kỹ thuật gồm decode_cold giữa B và C cùng thumbnail_cold giữa C và D, kết hợp 9 lượt so sánh JPEG, PNG, WEBP và hai ca TC04_format_PNG, TC04_format_WEBP.	Đo mức giảm đỉnh bộ nhớ PSS và thời gian giải mã khi hạ mẫu ảnh gốc 3000px theo khung hiển thị và khi chuyển sang ảnh thu nhỏ 720px.
TC05	Tắt mạng khi mở danh sách	Ngắt kết nối máy chủ bằng cách gỡ cầu nối adb reverse, sau đó chạy TC05_RAM_offline, TC05_disk_offline, TC05_uncached_offline, TC09_clear_offline và TC09_recover_network.	Ảnh đã lưu trong RAM hoặc đĩa vẫn hiển thị đủ 5/5 ảnh; vùng chưa lưu đệm ghi nhận đúng lỗi ERROR và hiển thị ảnh dự phòng mà không gây dừng ứng dụng.
TC06	URL ảnh lỗi hoặc máy chủ chậm	Chạy ca TC06_404 chèn lỗi HTTP 404 ở vị trí chia hết cho 10, ca TC06_slow_2s với độ trễ 2.000 ms và ca TC06_timeout với độ trễ 35.000 ms.	Giao diện không bị treo ANR; mục thứ 10 hiển thị ảnh lỗi trong khi 5 ảnh hợp lệ tải thành công; ca trễ 35s ngắt đúng tại ngưỡng readTimeout khoảng 15.1s.

Mã TC	Tên kịch bản bắt buộc	Cách hiện thực và mã ca chạy trong hồ sơ Cyclone 3.2	Chỉ số thu thập và kỳ vọng kiểm chứng
TC07	Tăng quy mô từ 100 lên 500 và 1.000 ảnh	Chạy ma trận đo ở cả 3 quy mô 100, 500 và 1.000 mục trên hai chế độ tải mới và cuộn cho A và D, kèm ca cuộn sâu D_traverse_1000 tới vị trí cuối 999.	Đánh giá biến thiên thời gian tải, PSS và độ trễ khung hình; nhờ tải theo nhu cầu nên khung nhìn đầu của 1.000 mục có mức PSS tương đương 100 mục.
TC08	So sánh trước và sau tối ưu	Tổng hợp ma trận đối chứng giữa A và D gồm 36 lượt tải mới và cuộn, 6 lượt tải lại, 24 lượt đo tách kỹ thuật và 2 bản dấu vết lấy mẫu CPU ART.	Xác định tỷ lệ phần trăm cải thiện của từng chỉ số kèm khoảng phân tán [Min; Max] qua các lượt đo và phân tích khách quan các đánh đổi kỹ thuật.

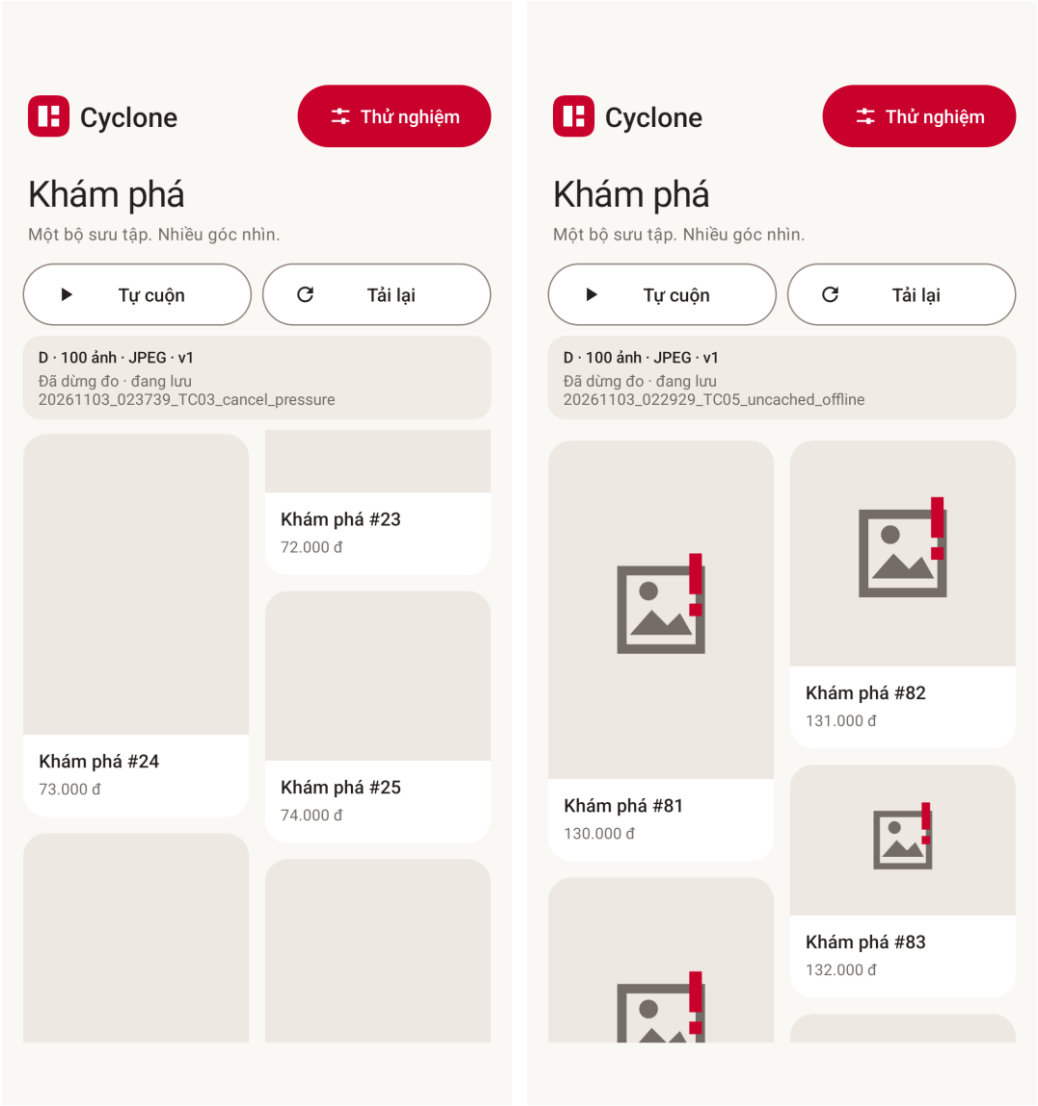
3.3 Giao diện và trạng thái xử lý ảnh

Các ảnh dưới đây được chụp trong các ca thực nghiệm Cyclone 3.2 trên máy ảo. Ảnh chụp minh họa kết quả chức năng; thời gian hiển thị và hiệu năng lấy từ log đo, không suy từ thời điểm chụp.



Hình 3.1 Danh sách ảnh đã tải thành công (trái) và màn hình chi tiết đúng sản phẩm (phải).

Placeholder và ảnh thay thế khi không tải được nguồn thử.



Hình 3.2 Placeholder ở ca hủy yêu cầu khi cuộn (trái) và ảnh lỗi ở vùng chưa cache khi ngắt nguồn thử (phải).

Trong ca stress, các yêu cầu có thể bị hủy khi ô rời màn hình; placeholder trong ảnh không được xem là yêu cầu tải thành công. Khi nguồn thử mất kết nối, các ảnh chưa cache chuyển sang ảnh lỗi, còn các ca cache RAM và đĩa được kiểm tra riêng ở Chương 4.

CHAPTER 4. EXPERIMENTAL RESULTS AND BENCHMARK ANALYSIS

4.1 Số liệu đo kiểm hiệu năng trước và sau tối ưu trên 36 lượt tải mới và 6 lượt tải lại

Toàn bộ phép đo dưới đây được thực thi trên máy ảo Pixel_5 AVD chạy Android 12 mức API 31 với định danh sdk_gphone64_x86_64 trên cổng emulator-5554, sử dụng bản dựng Cyclone-3.2-benchmark.apk có versionCode 5 và máy chủ HTTP cục bộ. Bảng 4.1 trình bày giá trị trung vị kèm khoảng phân tán [Min; Max] qua n = 3 lượt đo cho từng kịch bản, trích xuất chính xác từ tệp comparison.csv và báo cáo số liệu thực nghiệm:

Cách đọc bảng: nVP là số lượt đạt mốc viewport trên tổng số 3 lượt. Dấu “Không quan sát” nghĩa là vùng nhìn bị cuộn qua trước khi đủ điều kiện ghi mốc; đây là dữ liệu thiếu của chỉ số thời gian, không phải trạng thái chưa thực hiện ca thử. Không tính tỷ lệ cải thiện viewport khi A hoặc D không có số đo tương ứng.

Request và Decode là trung vị theo yêu cầu trong mỗi lượt rồi tổng hợp trung vị giữa các lượt; PSS đỉnh là trung vị của giá trị lớn nhất lấy mẫu từng lượt. Frame p90 là trung vị của phân vị 90 từng lượt; tỷ lệ trễ hạn là trung vị các tỷ lệ trên số frame có DEADLINE hợp lệ, không phải tỷ lệ suy ra bằng cách cộng mọi mẫu của ba lượt.

Dung lượng tệp trong bảng là tổng kích thước tệp của các yêu cầu NETWORK thành công, đối chiếu bộ dữ liệu cố định. Chỉ số này không bao gồm header, truyền lại, dữ liệu của yêu cầu bị hủy hoặc các chi phí TCP/TLS; vì vậy không được đồng nhất với lượng byte mạng thực đo trên đường truyền.

Kịch bản / Quy mô	Cấu hình	nVP	Viewport Pre-draw ms [Min; Max]	Request ms	Decode ms	PSS Đỉnh MiB
load / 100	A	3/3	1,057.00 [1027; 1216]	826.00	759.00	173.45
load / 100	D	3/3	739.00 [705; 809] Giảm 30.09%	612.00	485.00	76.17 Giảm 56.09%

Kịch bản / Quy mô	Cấu hình	nVP	Viewport Pre-draw ms [Min; Max]	Request ms	Decode ms	PSS Đỉnh MiB
load / 500	A	3/3	1,213.00 [1201; 1390]	926.00	880.00	160.30
load / 500	D	3/3	774.00 [736; 882] Giảm 36.19%	586.00	488.00	78.05 Giảm 51.31%
load / 1000	A	3/3	1,419.00 [1361; 1436]	1,118.00	1,055.00	155.78
load / 1000	D	3/3	870.00 [848; 872] Giảm 38.69%	761.00	560.00	76.01 Giảm 51.21%
reload / 100	A	3/3	1,361.00 [1302; 1384]	1,050.00	937.00	175.50
reload / 100	D	3/3	61.00 [56; 99] Giảm 95.52%	0.00	Trúng RAM	74.55 Giảm 57.52%
scroll / 100	A	0/3	Không quan sát	338.00	244.00	170.54
scroll / 100	D	3/3	775.00 [615; 775]	96.00	43.00	77.37 Giảm 54.63%
scroll / 500	A	0/3	Không quan sát	383.00	306.00	155.31
scroll / 500	D	3/3	987.00 [945; 1087]	148.00	71.00	78.96 Giảm 49.16%
scroll / 1000	A	0/3	Không quan sát	433.00	334.00	164.82
scroll / 1000	D	2/3	1,058.50 [938; 1179]	154.00	82.00	85.52 Giảm 48.11%

Bảng số liệu tiếp theo:

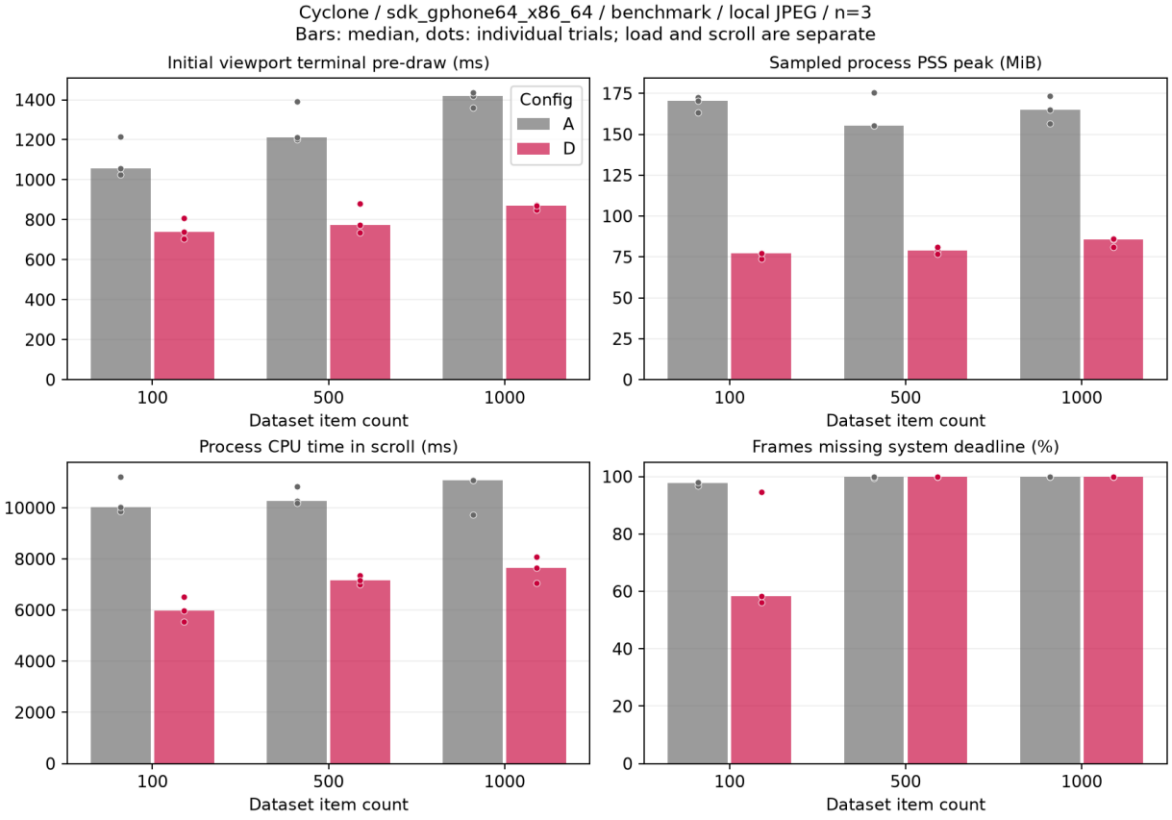
Kịch bản / Quy mô	Cấu hình	nVP	Process CPU ms	Frame p90 ms [Min; Max]	Tỷ lệ trễ hạn	GC Lần / ms	Dung lượng tệp byte
load / 100	A	3/3	2,317.00	212.19 [152.50; 284.63]	62.50%	1.0 / 121.0	1,256,649
load / 100	D	3/3	1,631.00 Giảm 29.61%	217.48 [210.6; 263.7] Tăng 2.49%	46.67%	1.0 / 151.0	182,216 Giảm 85.50%
load / 500	A	3/3	2,548.00	335.51 [183.01; 606.77]	100.00%	1.0 / 90.0	1,256,649

Kịch bản / Quy mô	Cấu hình	nVP	Process CPU ms	Frame p90 ms [Min; Max]	Tỷ lệ trễ hạn	GC Lần / ms	Dung lượng tệp byte
load / 500	D	3/3	1,822.00 Giảm 28.49%	503.02 [339.3; 678.1] Tăng 49.92%	83.33%	1.0 / 41.0	182,216
load / 1000	A	3/3	2,542.00	211.19 [198.60; 303.42]	100.00%	1.0 / 229.0	1,256,649
load / 1000	D	3/3	1,933.00 Giảm 23.96%	386.37 [324.1; 617.9] Tăng 82.95%	100.00%	1.0 / 35.0	182,216
reload / 100	A	3/3	2,451.00	817.68 [701.59; 884.52]	100.00%	1.0 / 205.0	1,256,649
reload / 100	D	3/3	422.00 Giảm 82.78%	187.56 [163.5; 197.8] Giảm 77.06%	100.00%	0.0 / 0.0	0 Tiết kiệm 100%
scroll / 100	A	0/3	10,029.00	160.56 [150.64; 207.70]	97.82%	13.0 / 650.0	6,924,386
scroll / 100	D	3/3	5,972.00 Giảm 40.45%	56.42 [51.04; 68.49] Giảm 64.86%	58.31%	1.0 / 107.0	1,012,969
scroll / 500	A	0/3	10,257.00	226.79 [210.47; 234.44]	100.00%	12.0 / 756.0	5,875,066
scroll / 500	D	3/3	7,154.00 Giảm 30.25%	92.00 [90.82; 103.36] Giảm 59.43%	100.00%	1.0 / 54.0	1,012,969
scroll / 1000	A	0/3	11,064.00	322.09 [210.98; 394.47]	100.00%	11.0 / 1133.0	4,771,026
scroll / 1000	D	2/3	7,656.00 Giảm 30.80%	98.76 [90.09; 150.60] Giảm 69.34%	100.00%	1.0 / 102.0	1,012,969

Số liệu chi tiết các pha xử lý khung hình ở phân vị thứ 90 và tổng số byte cấp phát:

- Kịch bản cuộn scroll / 100: Cấu hình A cấp phát 6,587,096 byte với layout p90 bằng 1.138 ms, draw p90 bằng 1.437 ms, sync p90 bằng 1.162 ms và 718 / 736 khung hình trễ hạn qua 3 lượt. Cấu hình D cấp phát 5,734,952 byte, giảm mạnh xuống layout p90 bằng 0.196 ms, draw p90 bằng 0.487 ms, sync p90 bằng 0.760 ms với 826 / 1213 khung hình trễ hạn.

- Kịch bản cuộn scroll / 500: Cấu hình A ghi nhận layout p90 bằng 2.146 ms, draw p90 bằng 1.471 ms, sync p90 bằng 1.375 ms với 551 / 552 khung hình trễ hạn. Cấu hình D giảm xuống layout p90 bằng 0.412 ms, draw p90 bằng 0.885 ms, sync p90 bằng 1.160 ms với 779 / 779 khung hình trễ hạn.
- Kịch bản cuộn scroll / 1000: Cấu hình A ghi nhận layout p90 bằng 3.531 ms, draw p90 bằng 1.828 ms, sync p90 bằng 1.526 ms với 424 / 424 khung hình trễ hạn. Cấu hình D giảm xuống layout p90 bằng 0.460 ms, draw p90 bằng 0.662 ms, sync p90 bằng 1.059 ms với 712 / 712 khung hình trễ hạn.



Hình 4.1: Biểu đồ trực quan so sánh hiệu năng giữa cấu hình cơ sở A và cấu hình tối ưu D trên máy ảo Pixel_5 API 31 với bản dựng benchmark 3.2, định dạng JPEG cục bộ và n = 3 lượt đo cho mỗi cấu hình.

4.2 Đo tách riêng từng kỹ thuật trên 24 lượt và so sánh định dạng JPEG, PNG, WebP trên 9 lượt

Do việc chuyển trực tiếp từ cấu hình A sang D thay đổi đồng thời nhiều yếu tố gồm bộ nhớ đệm, giải mã thu nhỏ và ảnh thu nhỏ, nghiên cứu đã thực hiện 24 lượt đo tách kỹ thuật trên quy mô 100 mục với n = 3 lượt đo mỗi cấu hình nhằm định lượng độc lập từng giải pháp:

Kịch bản đo tách	Cấu hình	Viewport ms	Request ms	Decode ms	PSS Đỉnh MiB
cache_prime tải mồi	A vs B	A: 1,988.0 B: 1,852.0	A: 1,584.0 B: 1,492.0	A: 1,500.0 B: 1,380.0	A: 172.54 B: 171.44
cache_warm tách bộ đệm	A vs B	A: 1,397.0 B: 70.0	A: 1,174.0 B: 0.0	A: 1,111.0 B: Trúng RAM	A: 175.52 B: 73.70
decode_cold tách hạ mẫu	B vs C	B: 1,880.0 C: 1,550.0	B: 1,571.0 C: 1,391.0	B: 1,400.0 C: 1,018.0	B: 174.04 C: 77.41
thumbnail_cold tách ảnh nhỏ	C vs D	C: 1,506.0 D: 1,445.0	C: 1,370.0 D: 1,351.0	C: 981.0 D: 1,080.0	C: 72.08 D: 72.87

Bảng số liệu tiếp theo:

Kịch bản đo tách	Cấu hình	CPU ms	Frame p90 ms	Nguồn NET/DISK/RAM
cache_prime tải mồi	A vs B	A: 3,677.0 B: 3,696.0	A: 701.43 B: 486.84	A: 5/0/0 B: 5/0/0
cache_warm tách bộ đệm	A vs B	A: 2,331.0 B: 470.0	A: 779.61 B: 178.87	A: 5/0/0 B: 0/0/5
decode_cold tách hạ mẫu	B vs C	B: 3,487.0 C: 3,109.0	B: 684.23 C: 632.22	B: 5/0/0 C: 5/0/0
thumbnail_cold tách ảnh nhỏ	C vs D	C: 3,052.0 D: 2,921.0	C: 609.85 D: 674.33	C: 5/0/0 D: 5/0/0

Phân tích tác động riêng biệt của từng kỹ thuật tối ưu:

Kịch bản cache_prime ở trạng thái tải mới: Bước nạp mồi ban đầu từ trạng thái lạnh trước khi đo tải lại; cả A và B đều tải 5 ảnh gốc 3000px từ mạng và giải mã nguyên bản.

Kịch bản cache_warm cô lập bộ nhớ đệm giữa A và B: Khi mở lại danh sách, cấu hình B trúng toàn bộ 5/5 ảnh từ bộ nhớ đệm RAM, giúp giảm 94.99% thời gian hiển thị vùng nhìn đầu tiên, giảm 79.84% thời gian CPU tiến trình và giảm 58.01% đỉnh bộ nhớ PSS so với A.

Kịch bản decode_cold cô lập giải mã theo vùng hiển thị giữa B và C: Khi cùng tải ảnh gốc 3000px từ mạng nhưng cấu hình C giải mã thu nhỏ theo kích thước thực của ImageView, đỉnh bộ nhớ PSS giảm 55.52%, thời gian giải mã giảm 27.29% và thời gian hiển thị vùng nhìn giảm 17.55% so với B.

Kịch bản thumbnail_cold giữa C và D: Đổi URL từ ảnh gốc 3.000 px sang ảnh thu nhỏ 720 px giảm 85,50% dung lượng tệp nguồn, giảm 4,05% thời gian viewport và 4,29% thời gian CPU. Đồng thời, PSS tăng 1,10%, decode tăng 10,09% từ 981 lên 1.080 ms và frame p90 tăng 10,57% từ 609,85 lên 674,33 ms. Như vậy, giảm dung lượng tệp không bảo đảm cải thiện đồng thời mọi chỉ số; phép thử này chỉ gồm ba lượt mỗi cấu hình.

Bảng 4.2: Kết quả thực nghiệm so sánh 3 định dạng ảnh JPEG, PNG và WEBP trên cấu hình D với quy mô 100 mục ở trạng thái tải mới qua n = 3 lượt đo:

Định dạng	Số lượt n	Request trung vị ms	Decode trung vị ms	PSS Đỉnh MiB	Process CPU ms	Dung lượng tệp 5 ảnh đầu
JPEG	3	1,321.00	1,110.00	75.78	2,775.00	182,216 byte - 177.9 KiB
PNG	3	1,419.00	1,116.00	76.99	3,074.00	394,444 byte - 385.2 KiB
WEBP	3	1,426.00	1,169.00	76.89	3,296.00	99,914 byte - 97.6 KiB

Nhận xét và đánh đổi kỹ thuật giữa các định dạng ảnh:

Định dạng JPEG: Đạt thời gian hoàn tất yêu cầu, thời gian giải mã và thời gian CPU trung vị thấp nhất trong 3 định dạng ở phép thử này, với dung lượng tệp ở mức trung bình.

Định dạng PNG: Trong bộ dữ liệu này, dung lượng tệp bằng khoảng 2,16 lần JPEG; thời gian request và CPU quan sát được cũng cao hơn. Phép đo phản ánh tổng tác động của định dạng trong cùng môi trường, không tách riêng toàn bộ chi phí mạng và chi phí giải mã.

Định dạng WebP: Dung lượng tệp nhỏ nhất, giảm 45,17% so với JPEG và khoảng 74,67% so với PNG. Đổi lại, decode và CPU trung vị cao hơn JPEG. Cùng quality 88 giữa các bộ mã hóa không bảo đảm chất lượng thị giác bằng nhau.

4.3 Kết quả 14 ca kiểm thử chức năng, vòng đời 3.2, cuộn sâu và phân tích TC01 đến TC08

Bảng 4.3: Kết quả thực tế của 14 ca kiểm thử chức năng đơn lượt trong hồ sơ Cyclone 3.2:

Mã ca thử	Kết quả	OK / ERR / CANCEL	Nguồn NET-DISK-RAM	VP Thành công / Kết thúc ms	Ý nghĩa minh chứng thực nghiệm
TC01_D_cold	ĐẠT	5 / 0 / 0	5 - 0 - 0	576 / 576	Mở danh sách lần đầu từ trạng thái sạch: tải thành công 5 ảnh đầu từ NETWORK.
TC02_D_RAM	ĐẠT	5 / 0 / 0	0 - 0 - 5	68 / 68	Tải lại trong cùng tiến trình: trúng 100% bộ nhớ đệm RAM với 5/5 ảnh, hiển thị sau 68 ms.
TC02_D_disk	ĐẠT	5 / 0 / 0	0 - 5 - 0	397 / 397	Xóa RAM giữ bộ nhớ đệm đĩa rồi tải lại: đọc 5/5 ảnh từ đĩa và giải mã trong 397 ms.
TC10_D_restart	ĐẠT	5 / 0 / 0	0 - 5 - 0	660 / 660	Khởi động lại tiến trình với PID mới: mất RAM, khôi phục đủ 5/5 ảnh từ bộ nhớ đệm đĩa.
TC05_RAM_offline	ĐẠT	5 / 0 / 0	0 - 0 - 5	128 / 128	Ngắt mạng khi ảnh đã lưu trong RAM: hiển thị thành công 5/5 ảnh trong 128 ms.
TC05_disk_offline	ĐẠT	5 / 0 / 0	0 - 5 - 0	283 / 283	Ngắt mạng khi chỉ còn bộ nhớ đệm đĩa: đọc từ đĩa và hiển thị 5/5 ảnh trong 283 ms.
TC05_uncached_offline	ĐẠT	0 / 7 / 0	0 - 0 - 0	Lỗi / 152	Ngắt mạng ở vùng chưa lưu đệm: ghi nhận 7 lỗi ERROR và hiển thị ảnh lỗi sau 152 ms.
TC09_clear_offline	ĐẠT	0 / 5 / 0	0 - 0 - 0	Lỗi / 85	Xóa sạch bộ nhớ đệm khi ngắt mạng rồi tải lại: cả 5 yêu cầu báo lỗi ERROR sau 85 ms.
TC09_recover_network	ĐẠT	5 / 0 / 0	5 - 0 - 0	382 / 382	Khôi phục mạng sau lỗi: tự động tải lại thành công 5/5 ảnh từ NETWORK trong 382 ms.

Mã ca thử	Kết quả	OK / ERR / CANCEL	Nguồn NET-DISK-RAM	VP Thành công / Kết thúc ms	Ý nghĩa minh chứng thực nghiệm
TC06_404	ĐẠT	5 / 1 / 0	5 - 0 - 0	Có lỗi / 273	Bật cờ lỗi HTTP 404 ở mục thứ 10: 5 ảnh hợp lệ tải OK, đúng 1 mục lỗi hiện ảnh thay thế.
TC06_slow_2s	ĐẠT	5 / 0 / 0	5 - 0 - 0	2,335 / 2,335	Máy chủ trễ 2.000 ms mỗi yêu cầu: giao diện không treo, hoàn tất 5 ảnh ở mốc 2,335 ms.
TC06_timeout	ĐẠT	0 / 5 / 0	0 - 0 - 0	Quá hạn / 15,113	Máy chủ trễ 35.000 ms: OkHttp ngắt đúng ngưỡng readTimeout 15s, đạt 15,113 ms với 5 lỗi.
TC04_format_PNG	ĐẠT	5 / 0 / 0	5 - 0 - 0	410 / 410	Kiểm thử nạp định dạng PNG: tải mạng và giải mã thành công 5/5 ảnh trong 410 ms.
TC04_format_WEBP	ĐẠT	5 / 0 / 0	5 - 0 - 0	324 / 324	Kiểm thử nạp định dạng WebP: tải mạng và giải mã thành công 5/5 ảnh trong 324 ms.

Các kiểm chứng bổ sung trong phiên bản Cyclone 3.2 gồm vòng đời, hồi quy, chia sẻ tệp và lấy mẫu CPU:

Kiểm chứng vòng đời và chi tiết: Các assertion trong đợt đo 3.2 đạt yêu cầu đối với ca hủy tiến trình khi ở nền, khôi phục với PID mới, giữ tên và tọa độ ô, xoay màn hình ngang dọc và mở đúng productId. Cấu hình D tải ảnh chi tiết 1.600 px thành công, không phát sinh URL raw 3.000 px trong ca đã kiểm tra.

Kiểm chứng cuộn sâu: Ca D_traverse_1000 chạy 66.040 ms, tới vị trí 999 tính từ 0 rồi quay lại đỉnh. Bản ghi có 998 ID phát sinh yêu cầu, 1.505 yêu cầu OK và 300 CANCEL; một ID có thể xuất hiện trong nhiều yêu cầu. Kết quả xác nhận tiến triển cuộn và hủy tác vụ trong ca thử, không đồng nghĩa 1.000 ảnh đều đã tải xong.

Kiểm chứng hồi quy và chia sẻ: Các điều kiện mở đúng chi tiết, quay lại giữ vị trí, xóa RAM rồi đọc đĩa, xuất CSV và cập nhật nội dung bằng version URL đều đạt trong các ca tương ứng. Ứng dụng nhận độc lập probe có UID khác Cyclone đã đọc được ZIP qua FileProvider và đối chiếu SHA-256 khớp.

4.4 Phân tích chi phí xử lý và các đánh đổi hiệu năng

Hai trace ART sampling có 9.828 bản ghi ở A và 11.835 bản ghi ở D, đều không overflow. Trong mẫu, số bản ghi chuyển tiếp hàm BitmapFactory/ImageDecoder trên main bằng 0; các luồng khác có 80 bản ghi ở A và 112 ở D. Kết quả phù hợp với cách sử dụng Coil bất đồng bộ. Đây là dữ liệu lấy mẫu, không phải số lần gọi hàm thực và không chứng minh mọi lần decode đều đã được quan sát.

Đối chiếu 10 frame có TOTAL_DURATION lớn nhất trong mỗi lượt scroll/1.000 cho thấy tổng thời gian layout, draw và sync chỉ chiếm 0,62% đến 2,69% tổng thời gian các frame được chọn. Giá trị theo lượt r1_A, r1_D, r2_A, r2_D, r3_A, r3_D lần lượt là 1,42%; 2,07%; 0,64%; 0,62%; 0,75%; 2,69%. Phần lớn thời gian của các frame chậm này nằm ngoài ba pha đã ghi. Dữ liệu hiện có không xác định riêng phần còn lại thuộc chờ CPU, GPU hay pha khác.

Ở kịch bản tải mới đứng yên, mỗi lượt chỉ ghi nhận khoảng 4 đến 17 frame; p90 vì vậy nhạy với một vài frame dài. Trong ma trận cuộn, p90 của D giảm 59,43% đến 69,34% so với A, nhưng tỷ lệ trễ hạn ở quy mô 500 và 1.000 vẫn có trung vị 100%. Cải thiện p90 cho thấy giảm độ dài của nhiều frame chậm trong mẫu, không có nghĩa giao diện đã đạt mượt hoàn toàn. Cỡ mẫu nhỏ và môi trường máy ảo là yếu tố cần xét khi diễn giải, không được dùng để khẳng định nguyên nhân của mọi chênh lệch.

Phân tích theo kỹ thuật: Cặp B/C giữ cùng URL ảnh gốc nhưng thay kích thước giải mã, làm PSS giảm từ 174,04 xuống 77,41 MiB và decode giảm từ 1.400 xuống 1.018 ms. Đây là bằng chứng trực tiếp trong thiết kế đối chứng cho thấy giải mã nguyên kích thước tạo chi phí bộ nhớ và xử lý lớn. Cặp A/B khi tải lại giảm viewport từ 1.397 xuống 70 ms nhờ 5/5 cache RAM hit, thể hiện phần tải và giải mã lặp được loại bỏ khi có bộ đệm.

Đối với cấp phát và thu gom rác, ma trận cuộn ghi nhận trung vị số lần GC giảm từ 11–13 ở A xuống 1 ở D, cùng với giảm PSS và CPU. Các chỉ số đồng biến này cho thấy giảm áp lực tài nguyên sau tối ưu. Tổng gc_ms gồm hoạt động GC đồng thời; không lấy tổng này làm thời gian dừng giao diện hoặc quy nguyên nhân cho từng frame.

Đối với cập nhật giao diện, layout p90 ở scroll/1.000 giảm từ 3,531 xuống 0,460 ms; draw p90 giảm từ 1,828 xuống 0,662 ms và sync p90 giảm từ 1,526 xuống 1,059 ms. Kết quả nhất quán với việc bớt yêu cầu layout và dùng ảnh phù hợp, nhưng A/D thay đổi nhiều yếu tố nên không dùng riêng cặp này để quy toàn bộ cải thiện cho một thay đổi mã.

Từ các phép đối chiếu, giải pháp ưu tiên cache cho tải lại, hạ mẫu ngay khi giải mã và ảnh thu nhỏ cho danh sách. Các kỹ thuật này đã cải thiện thời gian chuẩn bị hiển thị, PSS và CPU trong các ca tương ứng. Việc đánh giá độ mượt đồng thời xem xét p90, tỷ lệ trễ hạn, số frame quan sát và các đánh đổi, thay vì chỉ chọn chỉ số có lợi.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Kết luận và đóng góp của đề tài

Đề tài đã xây dựng ứng dụng thực nghiệm Cyclone 3.2 bằng Kotlin, XML và RecyclerView; hoàn thành các phép so sánh tải mới, tải lại, cache, kích thước giải mã, ảnh thu nhỏ và định dạng ảnh trên máy ảo Pixel 5 API 31. Kết quả cho phép đánh giá hiệu quả và đánh đổi của từng nhóm giải pháp trong điều kiện thử nghiệm đã kiểm soát:

1. Về lý thuyết: Báo cáo hệ thống hóa chu trình tải ảnh, Kotlin Coroutines, cache RAM/đĩa/HTTP, Bitmap, giải mã, lazy loading, tái sử dụng ViewHolder và phương pháp đo hiệu năng; liên hệ các cơ chế với phần triển khai trong Cyclone.
2. Về triển khai: Ứng dụng hỗ trợ danh sách 100, 500 và 1.000 mục, xem chi tiết, placeholder, ảnh lỗi, hủy yêu cầu cũ và khôi phục trạng thái. Năm cấu hình A đến E dùng chung giao diện và bộ dữ liệu để tái tạo các đối chiếu.
3. Về thực nghiệm: Hồ sơ được lập chỉ mục gồm 114 phiên; ma trận chính có 36 lượt cold A/D, 6 lượt tải lại sau 6 lượt nạp mồi, 24 lượt tách kỹ thuật và 9 lượt định dạng. Các nhóm kiểm thử và lấy mẫu CPU được giữ riêng.

- Khi mở danh sách lần đầu ở quy mô 100, 500 và 1.000 mục, cấu hình tối ưu D giảm từ 30.09% đến 38.69% thời gian chuẩn bị vẽ vùng nhìn đầu tiên, giảm từ 51.21% đến 56.09% đỉnh bộ nhớ PSS lấy mẫu từ mức 155–173 MiB xuống 76–78 MiB, giảm từ 23.96% đến 29.61% thời gian CPU tiến trình và giảm 85.50% dung lượng tệp tải về.
- Khi tải lại danh sách 100 mục, nhờ trúng 100% bộ nhớ đệm RAM, cấu hình D giảm 95.52% thời gian hiển thị vùng nhìn từ 1,361.00 ms xuống 61.00 ms và giảm 82.78% thời gian CPU.
- Khi cuộn danh sách ở quy mô 100, 500 và 1.000 mục, cấu hình D giảm từ 59.43% đến 69.34% độ trễ khung hình frame_p90_ms, giảm từ 30.25% đến 40.45% thời gian CPU và giảm số đợt thu gom rác ART GC từ 11–13 lần xuống còn 1 lần.

5.2 Phạm vi áp dụng và hướng phát triển

Các kết quả trong báo cáo áp dụng cho máy ảo Pixel 5 chạy Android 12, API 31, với nguồn ảnh cục bộ, bộ dữ liệu cố định và ba lượt lặp mỗi cấu hình. PSS là đỉnh lấy mẫu; viewport là mốc pre-draw; dữ liệu ART là sampling. Những ranh giới này được giữ khi diễn giải kết quả, đặc biệt khi đánh giá frame trễ và chi phí GC.

Hướng phát triển là tăng số lần lặp, mở rộng nội dung ảnh gần với sử dụng thực tế và đồng bộ thêm dữ liệu CPU/GPU với từng khoảng frame để phân tích sâu hơn phần thời gian còn lại. Các mở rộng như xác thực cache bằng ETag và đo bằng Macrobenchmark có thể được khảo sát trên nền ứng dụng hiện có.

TÀI LIỆU THAM KHẢO

1. Google Android Developers. Manage memory and Loading large bitmaps efficiently. developer.android.com/topic/performance/graphics
2. Google Android Developers. Guide to app architecture và RecyclerView. developer.android.com/topic/architecture; developer.android.com/develop/ui/views/layout/recyclerview
3. Google Android Developers. Kotlin coroutines best practices. developer.android.com/kotlin/coroutines/coroutines-best-practices
4. Google Android Developers. FrameMetrics và Debug API. developer.android.com/reference/android/view/FrameMetrics; developer.android.com/reference/android/os/Debug
5. Coil Contributors. Coil 2.7.0. github.com/coil-kt/coil/tree/2.7.0
6. Square. OkHttp. square.github.io/okhttp
7. R. Fielding, M. Nottingham và J. Reschke. HTTP Caching, RFC 9111, 2022. rfc-editor.org/rfc/rfc9111.html
8. Google Developers. WebP image format. developers.google.com/speed/webp
9. Tài liệu học phần Phát triển ứng dụng di động: Kotlin basics, Functions, Classes and objects; Build your first Android app; Layouts; App navigation; App architecture UI Layer và Persistence; Android RecyclerView; các bài thực hành Kotlin và XML layouts.